# C PROGRAMMING

*Lesson 13 — Variable Scope*

Based on: Bro Code C Tutorial Series

## 1. What is Variable Scope?

Scope refers to where in the program a variable exists and can be accessed. A variable only lives within the block it was declared in. Once that block ends, the variable is gone.

Think of each set of curly braces { } as a room. A variable declared inside a room only exists in that room — it cannot be seen or used outside.

```
void myFunction() {
    int x = 10;    // x lives inside myFunction only
}                  // x is destroyed here

int main() {
    printf("%d\n", x);    // ERROR -- x does not exist here
    return 0;
}
```

## 2. Local Scope

A local variable is declared inside a function or block { }. It only exists within that block and is destroyed when the block ends.

### Same name in different functions — allowed

```
int add(int x, int y) {
    int result = x + y;    // this "result" belongs to add()
    return result;
}

int main() {
    int result = add(3, 4);    // this "result" belongs to main()
    printf("%d\n", result);    // Output: 7
    return 0;
}

// Both variables are named "result" but they are completely
// separate -- they live in different scopes.
```

> **Key Point:** Two variables can share the same name if they are in different scopes. They are completely separate variables that just happen to have the same name.

### Same name in the same scope — NOT allowed

```
int main() {
    int result = 10;
    int result = 20;   // ERROR: redefinition of "result"
    return 0;
}
```

> **Rule:** You cannot declare two variables with the same name inside the same scope. This causes a compiler error.

## 3. Functions Cannot See Each Other's Variables

Each function is its own isolated scope. The only way to share data between functions is by passing arguments.

```
void subtract() {
    printf("%d\n", x - y);   // ERROR -- x and y are unknown here
}

int main() {
    int x = 10;
    int y = 3;
    subtract();   // does not pass x or y!
    return 0;
}

// Fix: pass x and y as arguments
void subtract(int x, int y) {
    printf("%d\n", x - y);   // works correctly
}

int main() {
    int x = 10;
    int y = 3;
    subtract(x, y);   // pass the values
    return 0;
}
```

## 4. Global Scope

A global variable is declared outside all functions. It can be accessed and modified by any function in the program.

```
#include <stdio.h>

int result = 0;   // global variable -- accessible everywhere

void add(int x, int y) {
    result = x + y;   // can access global result
}

int main() {
    add(3, 4);
    printf("%d\n", result);   // Output: 7
    return 0;
}
```

**Avoid Global Variables:** Global variables seem convenient but they are dangerous. Any function can accidentally change them, making bugs very hard to track down. Always prefer passing arguments instead.

## 5. Block Scope Inside Functions

Scope is not just about functions — any set of { } creates its own scope, including if blocks, loops, and switch cases.

```
int main() {
    int x = 10;

    if (x > 5) {
        int y = 20;            // y only exists inside this if block
        printf("%d\n", y);    // works fine
    }

    printf("%d\n", y);        // ERROR -- y no longer exists here
    return 0;
}
```

**Remember:** Variables declared inside if blocks, loops, or switch cases only exist within those { } braces. This is the same rule as functions.

## 6. Practice Problem

**Practice: Scope Detective**

This problem tests whether you truly understand scope. Without running the code, predict the output or identify the error for each snippet.

### Snippet 1 — What is the output?

```
int x = 5;

void doubleIt() {
    int x = 10;
    printf("%d\n", x);
}

int main() {
    doubleIt();
    printf("%d\n", x);
    return 0;
}
```

### Snippet 2 — What is the error?

```
int main() {
    if (1 == 1) {
        int secret = 42;
    }
    printf("%d\n", secret);
    return 0;
}
```

### Snippet 3 — Fix this code

```
void multiply() {
    printf("%d\n", a * b);
}

int main() {
    int a = 6;
    int b = 7;
    multiply();
    return 0;
}
```

> **Hint:** For Snippet 1 think carefully about which x each function sees. For Snippet 2 think about where secret is declared. For Snippet 3 think about how to share variables between functions.